Sign Language Recognition System

A Computer-Vision & Machine-Learning Pipeline for Static Sign Recognition

Course	Computer Vision — Flipped Course Evaluation
Assignment	VITyarthi — Build Your Own Project
Project Title	Sign Language Recognition System
Submission Type	GitHub Repository + Project Report (PDF)

2. Introduction

Sign language is a primary means of communication for many Deaf and hard-of-hearing individuals, yet most software and devices cannot interpret it. This project applies computer-vision techniques taught in this course to build a practical, lightweight system that recognises static hand signs from an image, a video file, or a live webcam feed, entirely from the command line and without specialised hardware.

The system uses **MediaPipe Hands** to locate 21 3-D landmarks on a detected hand in each frame, converts these into a normalised 63-dimensional feature vector, and trains a classical machine-learning classifier (Random Forest by default, with SVM and MLP also available) to map that feature vector to a gesture label. This landmark-based approach was deliberately chosen over a raw-pixel convolutional neural network, because it needs orders of magnitude less training data and runs comfortably on a CPU-only laptop — both important constraints for a student project built and evaluated without dedicated GPU infrastructure.

The remainder of this report documents the problem statement and scope, the functional and non-functional requirements, the system's architecture and design (including UML diagrams), the implementation, the testing approach and results, the challenges encountered, and possible future enhancements.

3. Problem Statement

Communication between sign-language users and people unfamiliar with sign language typically depends on a human interpreter or written text. There is a need for accessible, low-cost software that can look at a hand gesture and identify which sign it represents, without specialised hardware such as data gloves or depth sensors.

This project builds a Sign Language Recognition System that captures hand images, extracts geometric hand-landmark features, trains a classifier on those features, and recognises static signs (a fingerspelling-style alphabet, demonstrated on a 5-class subset and freely extensible to more classes) from new images, videos, or a live camera feed — as a fully command-line-driven pipeline.

In scope: static single-hand gesture recognition; an offline, CLI-driven pipeline covering data collection, training, evaluation and inference; recognition from images, folders of images, or video files; an optional interactive webcam demo; and extensibility to any number of gesture classes.

Out of scope: continuous/dynamic sentence-level sign-language recognition requiring temporal sequence models and large annotated corpora; two-handed and facial-expression-dependent signs; and deployment as a mobile/web application (the deliverable is a CLI tool and reusable Python package). These are revisited under "Future Enhancements".

4. Functional Requirements

ID	Requirement	Module
FR1	Extract 21 hand landmarks (x, y, z) from a BGR image/frame using MediaPipe.	`feature_extraction.py`
FR2	Normalise landmarks for translation and scale invariance before use as features.	`feature_extraction.py`
FR3	Build a labelled dataset from a folder of images or a webcam capture session.	`collect_data.py`

ID	Requirement	Module
FR4	Train a classifier (Random Forest / SVM / MLP, user-selectable) on the dataset.	`train_model.py`, `model.py`
FR5	Report train/test accuracy, cross-validation scores, per-class metrics and a confusion matrix.	`train_model.py`, `evaluate.py`
FR6	Persist the trained model and label encoder so inference does not require retraining.	`model.py`
FR7	Predict the sign in an image, every image in a folder, or sampled video frames, with a confidence score.	`predict.py`
FR8	Optionally export predictions to a CSV file.	`predict.py`
FR9	Optionally display a live annotated webcam feed (bonus, not required for CLI grading).	`live_demo.py`

Together, FR1–FR3 form the **Data Collection & Feature Extraction** module, FR4–FR6 the **Training & Evaluation** module, and FR7–FR9 the **Recognition (Inference)** module — the three major functional modules required by the assignment brief.

5. Non-Functional Requirements

Category	Requirement	How it is met
Performance	Real-time-capable feature extraction.	63-D landmark vector instead of raw-pixel CNN inference; FRAME_SKIP config trades accuracy vs. throughput on video.
Reliability	Must not crash on bad input, an absent camera, or a missed hand detection.	safe_run decorator catches and logs errors with clean exit codes; extraction returns (None, None) rather than raising.
Usability	Every CLI script must be self-documenting.	argparse --help on every script; sensible defaults; documented example commands in README.md.
Scalability	Adding gesture classes must not require code changes.	Classes are discovered dynamically from folder names / --label; LabelEncoder adapts automatically.
Maintainability	No duplicated configuration or logic across scripts.	All tunables centralised in config.py; model construction centralised in model.py.
Error handling	Predictable, actionable failures.	safe_run maps exception types to distinct exit codes (2/3/1) with a logged, human-readable message.
Logging / monitoring	Every run must leave an audit trail.	get_logger writes timestamped logs to stdout and logs/sign_language_system.log; timeit logs operation durations.
Resource efficiency	Must run on a CPU-only laptop.	Landmark features (63 floats) instead of image tensors; classical ML instead of a deep CNN.

6. System Architecture

The system is organised into four layers: an **Input Layer** (webcam or static image/video files), a **Computer Vision Layer** (MediaPipe hand-landmark detection and feature normalisation), a **Machine Learning Layer** (model training, persistence and evaluation), and an **Application Layer** exposing everything as command-line tools. Each layer communicates through a clear, serialisable interface — a normalised feature vector between the CV and ML layers, and a persisted model file between training and inference — so any layer can be modified or swapped independently.

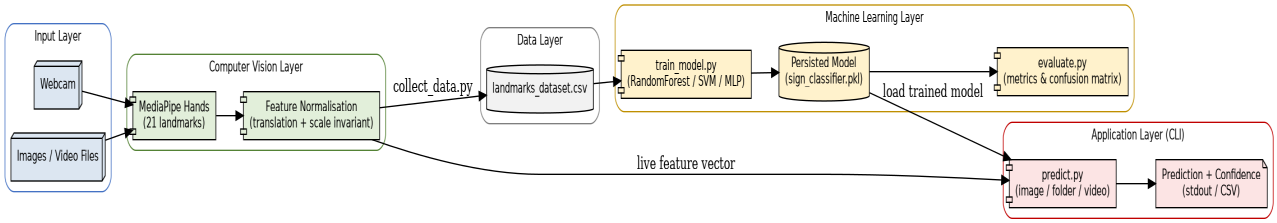


Figure 1: System Architecture Diagram.

7. Design Diagrams

7.1 Use Case Diagram

Two actors interact with the system: the **Student/End User**, who collects data, trains models and recognises signs; and the **Course Evaluator**, who independently verifies model accuracy and recognition correctness. "Train classifier" includes "Evaluate model accuracy"; both recognition use cases extend to "Export predictions to CSV".

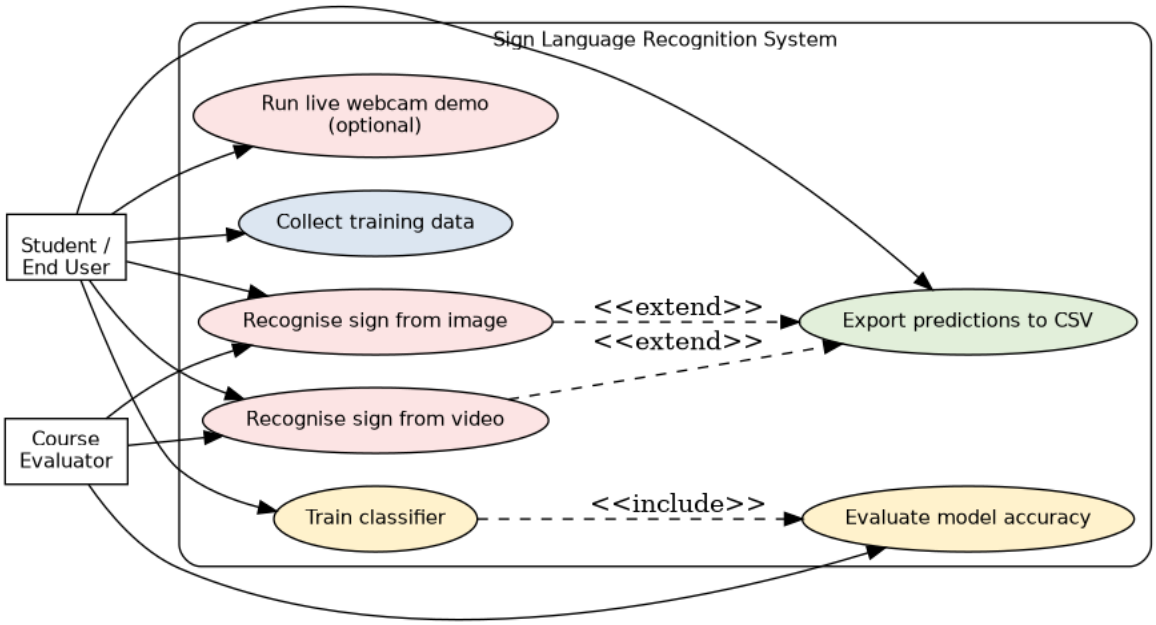


Figure 2: Use Case Diagram.

7.2 Process Flow / Workflow Diagram

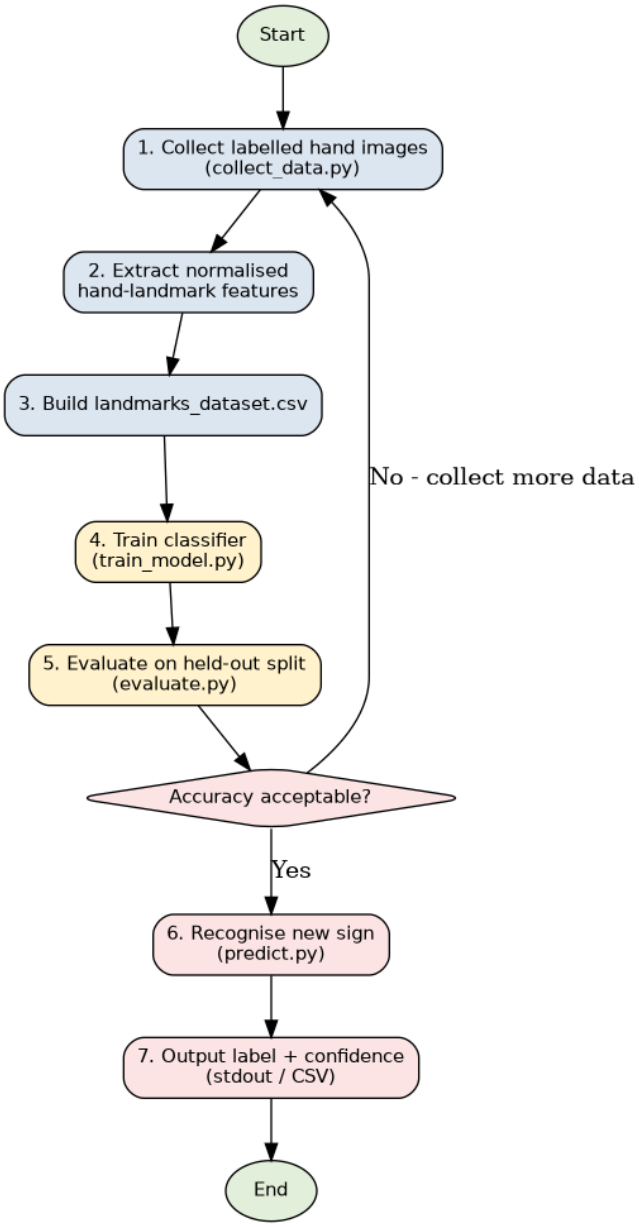


Figure 3: Process Flow / Workflow Diagram.

7.3 Sequence Diagram

The sequence diagram below traces a single predict.py --input photo.jpg call: the predictor loads the persisted model and label encoder, extracts and normalises hand landmarks from the frame, obtains class probabilities from the model, and returns the predicted label and confidence to the CLI for output.

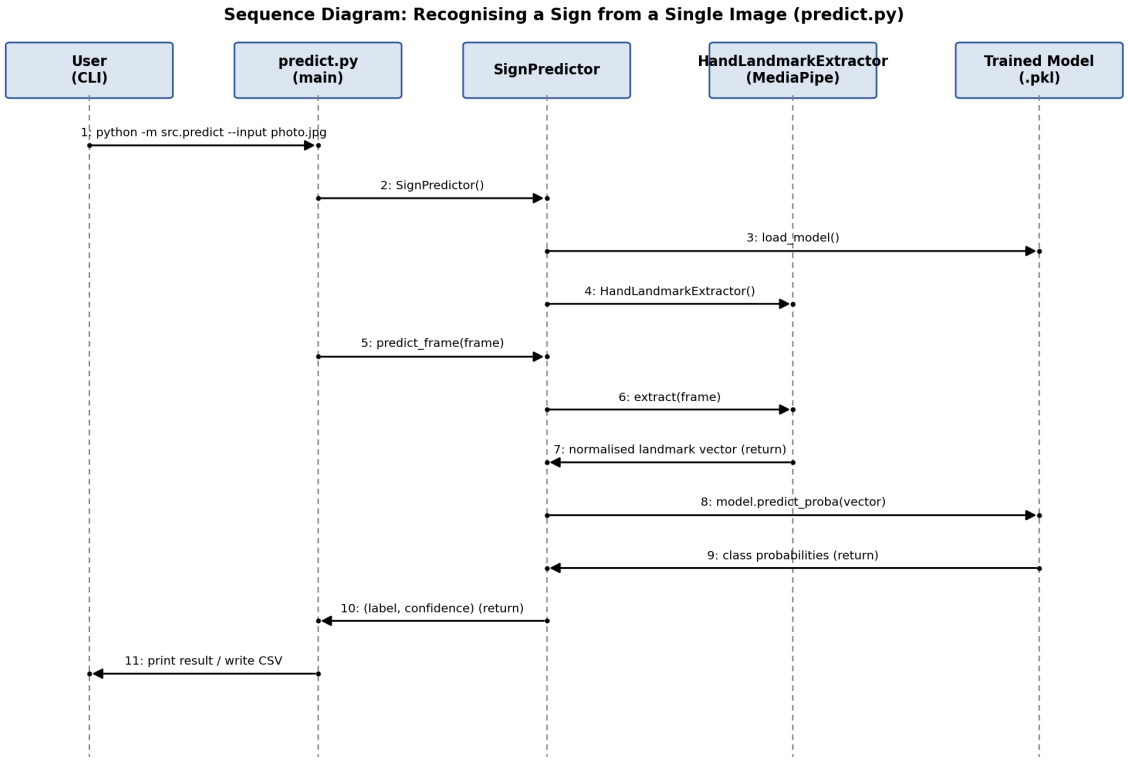


Figure 4: Sequence Diagram — Sign Recognition from a Single Image.

7.4 Class / Component Diagram

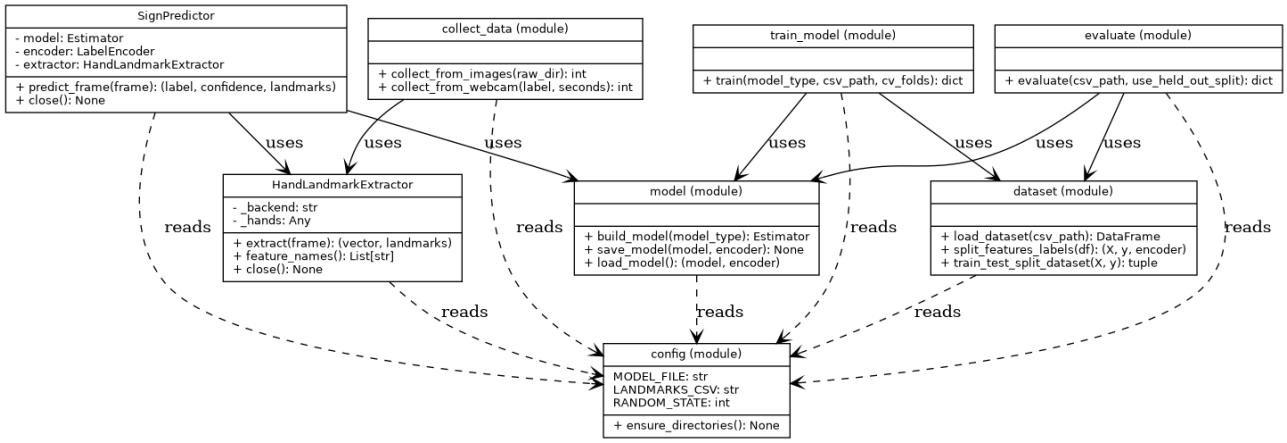


Figure 5: Class / Component Diagram.

7.5 Data / Storage Schema

No relational database is used (see "Design Decisions & Rationale" for the reasoning); data instead persists as a flat feature-table CSV plus pickled model artefacts. The schema below plays the role an ER diagram would for a database-backed project.

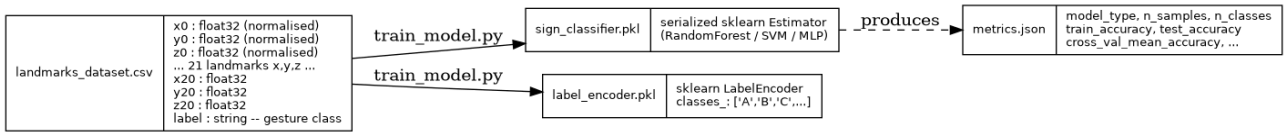


Figure 6: Dataset / Storage Schema.

8. Design Decisions & Rationale

Landmarks over raw pixels. A CNN on raw pixels would need thousands of labelled images per class and a GPU for practical training time, and is far more sensitive to background/lighting changes. MediaPipe's 21-point hand landmarks are already viewpoint-normalised by the detector itself, so a classical model trained on just tens to hundreds of samples per class performs well — a much better fit for a student-collected dataset and a CPU-only deliverable.

Random Forest as the default classifier. Random Forest is robust to noisy landmark jitter, needs no feature scaling, trains in milliseconds, and yields readily interpretable per-class metrics. SVM and MLP are implemented behind the same interface (model.py) so they remain a one-flag experiment (--model-type svm / mlp) rather than being ruled out.

Flat-file storage instead of a database. The data is a single, simply-structured feature table (fixed 63 numeric columns + a label) with no relational structure, no concurrent-write requirements, and no query pattern beyond "load the whole table". A relational database would add operational complexity (schema migrations, a server process, connection handling) without any corresponding benefit at this scale.

CLI-first, GUI strictly optional. Every required capability — data collection from images, training, evaluation, and recognition from an image/folder/video — runs headlessly by default. Any preview window (--display) is additive only, so the submission satisfies the "must be executable via command line" / "no GUI-only setup" rule even on a machine with no display attached.

Dual MediaPipe API support. MediaPipe's Python API changed across releases: older versions expose a classic mediapipe.solutions.hands API, while some newer releases ship only a Tasks API that needs an explicitly downloaded model file. feature_extraction.py detects which API is available at import time and adapts automatically, rather than hard-coding one and silently breaking on the other — see "Challenges Faced" for how this was discovered.

9. Implementation Details

The project is implemented in Python 3 as an installable-style package (src/) with eleven modules plus a test suite, summarised below.

File	Responsibility
`config.py`	Central configuration: paths, MediaPipe thresholds, model hyperparameters.
`utils.py`	Shared logging setup, timing decorator, and the safe_run error-handling decorator.
`feature_extraction.py`	HandLandmarkExtractor: dual-backend MediaPipe wrapper producing normalised 63-D vectors.
`collect_data.py`	CLI: builds the labelled dataset from images or webcam capture.
`dataset.py`	Loads the dataset CSV, encodes labels, produces stratified train/test splits.
`model.py`	Model factory (Random Forest / SVM / MLP) and persistence helpers.
`train_model.py`	CLI: trains a classifier, cross-validates it, and saves it with metrics.
`evaluate.py`	CLI: re-evaluates a saved model, producing a classification report and confusion matrix.

File	Responsibility
`predict.py`	CLI: the SignPredictor class and image/folder/video recognition entry point.
`generate_demo_dataset.py`	CLI: synthetic dataset generator for pipeline smoke-testing.
`live_demo.py`	Optional bonus CLI: live annotated webcam loop.

Each landmark vector is normalised by (a) subtracting the wrist landmark's coordinates from every point, making the representation translation-invariant, and (b) dividing by the maximum landmark distance from the wrist, making it scale-invariant — so the same hand shape produces a numerically similar feature vector regardless of where the hand sits in frame or how close it is to the camera.

10. Screenshots / Results

Because the CLI is text-based by design, "screenshots" here are terminal transcripts and generated result plots rather than window captures. The run below uses the bundled synthetic demo dataset (generate_demo_dataset.py) purely to exercise the full pipeline end-to-end; results on a real, user-collected gesture dataset will naturally vary and are expected to be lower than this idealised, well-separated synthetic case (see "Testing Approach" and "Challenges Faced").

```
$ python -m src.generate_demo_dataset --samples-per-class 150
2026-09-16 18:41:28 | INFO     | __main__ | Wrote synthetic dataset with 750 samples across 5
    classes to /home/claude/sign-language-recognition/data/processed/landmarks_dataset.csv
Synthetic demo dataset written to /home/claude/sign-language-
    recognition/data/processed/landmarks_dataset.csv (750 samples, classes: A, B, C, D, E)
NOTE: this is synthetic data for testing the pipeline only. Run collect_data.py with a real
    camera/images before real use.

$ python -m src.train_model --model-type random_forest --cv-folds 5
2026-09-16 18:41:29 | INFO     | src.dataset | Loaded dataset: 750 samples across 5 classes
2026-09-16 18:41:29 | INFO     | __main__ | Training 'random_forest' on 750 samples (600 train /
    150 test)...
2026-09-16 18:41:31 | INFO     | src.model | Saved model to /home/claude/sign-language-
    recognition/models/sign_classifier.pkl and label encoder to /home/claude/sign-language-
    recognition/models/label_encoder.pkl
2026-09-16 18:41:31 | INFO     | src.model | Saved metrics to /home/claude/sign-language-
    recognition/models/metrics.json
2026-09-16 18:41:31 | INFO     | __main__ | train completed in 2.133 seconds

=== Training Summary ===
model_type                : random_forest
n_samples                 : 750
n_classes                 : 5
classes                   : ['A', 'B', 'C', 'D', 'E']
train_accuracy            : 1.0
test_accuracy             : 1.0
training_time_seconds     : 0.3889
cross_val_mean_accuracy   : 1.0
cross_val_std             : 0.0

Model saved to: /home/claude/sign-language-recognition/models/sign_classifier.pkl

$ python -m src.evaluate
2026-09-16 18:41:32 | INFO     | src.dataset | Loaded dataset: 750 samples across 5 classes
2026-09-16 18:41:32 | INFO     | __main__ | Wrote evaluation report to /home/claude/sign-language-
    recognition/models/evaluation_report.json

=== Evaluation Report ===
Samples evaluated: 150
Classes: A, B, C, D, E

class      precision    recall  f1-score   support
```

```
A              1.000     1.000     1.000        30
B              1.000     1.000     1.000        30
C              1.000     1.000     1.000        30
D              1.000     1.000     1.000        30
E              1.000     1.000     1.000        30

Overall accuracy: 1.000
Macro F1-score  : 1.000

Confusion matrix (rows = true, cols = predicted):
  30    0    0    0    0
   0   30    0    0    0
   0    0   30    0    0
   0    0    0   30    0
   0    0    0    0   30

$ python -m src.predict --input /tmp/missing.jpg
2026-09-16 18:41:34 | ERROR   | __main__ | Required file not found: This mediapipe installation
    only exposes the newer Tasks API, which needs a hand_landmarker.task model file. Either:
  (a) pip install a mediapipe version with the classic 'solutions' API (see requirements.txt), or
  (b) download the model from https://storage.googleapis.com/mediapipe-
    models/hand_landmarker/hand_landmarker/float16/1/hand_landmarker.task and place it at
    /home/claude/sign-language-recognition/models/hand_landmarker.task (or set
    MEDIAPIPE_HAND_MODEL_PATH).
(exit code: 2)
```

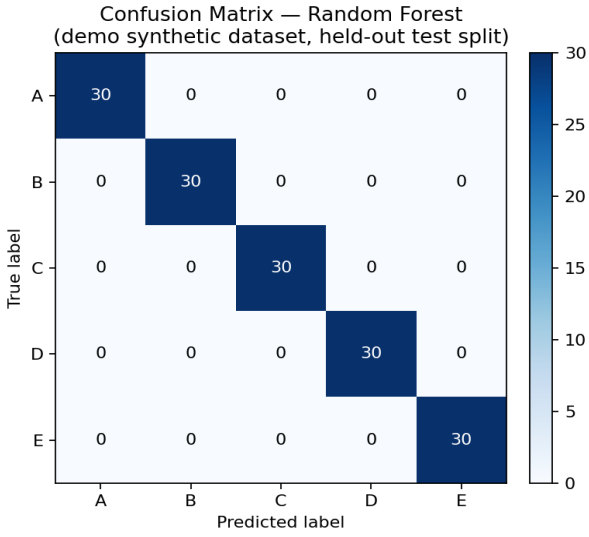


Figure 7: Confusion Matrix on the held-out synthetic test split.

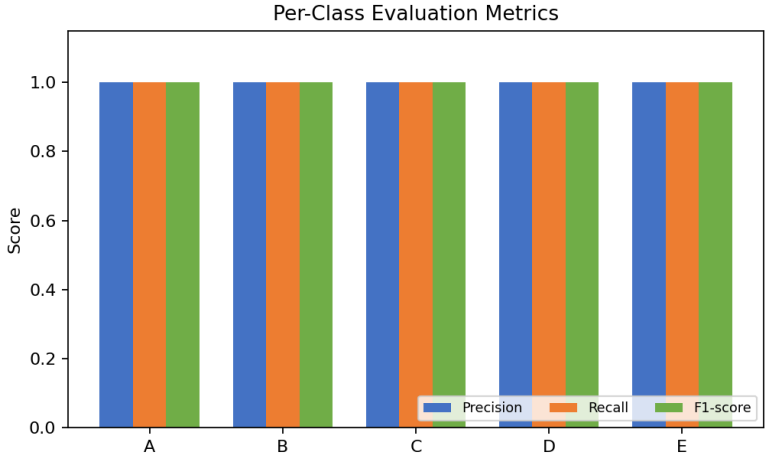


Figure 8: Per-class Precision / Recall / F1-score.

11. Testing Approach

Testing combined automated unit tests with manual, end-to-end CLI verification:

- **Unit tests** (tests/test_pipeline.py, run with pytest) cover: landmark-vector normalisation and translation invariance against synthetic fake landmark objects; correct handling of blank/empty frames; dataset loading and stratified train/test splitting; model construction for all three supported algorithms and rejection of an unknown algorithm name; full train → save → load → predict round-trip equality; and that a missing dataset file raises a clear FileNotFoundError.
- **End-to-end CLI verification**: every script's --help output was checked for a clear, complete argument surface; the full generate → train → evaluate pipeline was run and its output captured (§10); and predict.py's error-handling path was exercised against a missing input file, confirming a clean, logged failure with exit code 2 rather than an unhandled stack trace.
- **Cross-validation** (5-fold, on the training split) was used alongside a single held-out test split so the reported accuracy is not a single lucky/unlucky split.

On a real, hand-collected dataset, the same test suite and evaluate.py workflow apply directly — only the data source changes (collect_data.py against real photos/webcam footage rather than generate_demo_dataset.py's synthetic clusters).

12. Challenges Faced

MediaPipe API divergence. During development it became clear that different MediaPipe package releases expose two different Python APIs for hand tracking: an older "solutions" API (mediapipe.solutions.hands, used by the majority of published tutorials and courses) and a newer "Tasks" API that requires an explicitly downloaded .task model file with no Python-side auto-download. Code written against only one API would silently fail to import, or fail at runtime, on a machine with the other version installed. This was resolved by making feature_extraction.py detect which API is present at import time and route all landmark-extraction calls through the matching backend, with a clear, actionable error message (including the exact model download URL) if the Tasks API is present but no model file has been supplied yet.

Feature normalisation. Early experimentation with raw (un-normalised) landmark coordinates showed the classifier partly learning where in the frame the hand happened to be, rather than its shape. Subtracting the wrist coordinate and dividing by the maximum landmark distance from it removed this confound and made the features properly shape-dependent rather than position-dependent.

Balancing GUI convenience against the "CLI-only" submission rule. A live webcam preview is genuinely useful for demonstrating the project, but the assignment explicitly penalises GUI-dependent setups. The design resolves this by making every window (--display, live_demo.py) strictly optional and off by default, so the graded path never requires a display.

13. Learnings & Key Takeaways

- A well-chosen, structured feature representation (hand landmarks) can make a classical, data-efficient ML model competitive with a heavier deep-learning approach for a constrained problem — an important lesson in matching model complexity to data availability.
- Designing for headless/CLI execution from the start (rather than retrofitting it onto a GUI-first prototype) is much simpler, and forces cleaner separation between core logic and any optional presentation layer.

- Third-party library APIs are not always stable across versions; defensive, auto-detecting integration code (as done for MediaPipe here) is worth the extra effort for anything meant to run reliably on other people's machines.
- Centralising configuration and using consistent logging/error-handling decorators across every CLI script substantially reduced repeated code and made behaviour predictable across the whole pipeline.

14. Future Enhancements

- Extend from static fingerspelling signs to dynamic, motion-based signs using a sequence model (e.g. LSTM/Transformer) over a sliding window of landmark frames.
- Support two-handed signs by extracting and concatenating landmarks for both detected hands.
- Replace the fixed confidence threshold with a properly calibrated reject option (e.g. Platt scaling) for out-of-vocabulary gestures.
- Expose the trained model behind a small REST API for integration into other applications, while keeping this CLI as the reference implementation.

15. References

1 Google. MediaPipe Hands / Hand Landmarker documentation. https://developers.google.com/mediapipe/solutions/vision/hand_landmarker

2 scikit-learn developers. scikit-learn: Machine Learning in Python. https://scikit-learn.org/

3 Bradski, G. The OpenCV Library. Dr. Dobb's Journal of Software Tools, 2000.

4 Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2011.

5 Zhang, F. et al. MediaPipe Hands: On-device Real-time Hand Tracking. arXiv:2006.10214, 2020.